

IRepository<T>

● IRepository<T>

```
/// <summary>
/// 레포지토리 인터페이스
/// </summary>
/// <typeparam name="T"> </typeparam>
public interface IRepository<T> where T : class, new()
{
    /// <summary>
    /// 특정 ID에 해당하는 Entity를 비동기적으로 가져옵니다.
    /// </summary>
    /// <param name="id">검색하려는 Entity의 고유 식별자(ID)</param>
    /// <returns>
    /// 요청된 ID에 해당하는 Entity 객체를 반환합니다.
    /// 만약 해당 ID의 Entity가 없으면 null을 반환합니다.
    /// </returns>
    Task<T?> GetByIdAsync(int id);

    /// <summary>
    /// 모든 Entity를 비동기적으로 가져옵니다.
    /// </summary>
    /// <returns>
    /// 데이터베이스에 존재하는 모든 Entity를 컬렉션 형태로 반환합니다.
    /// Entity의 목록을 IEnumerable 반환하며, 비어 있을 수 있습니다.
    /// </returns>
    Task<IEnumerable<T>> GetAllAsync();

    /// <summary>
    /// 새로운 Entity를 비동기적으로 추가합니다.
    /// </summary>
    /// <param name="entity">추가할 Entity 객체</param>
    /// <remarks>
    /// 새로 추가하려는 Entity는 유효한 값이어야 하며,
    /// 해당 Entity의 기본 키(ID)는 자동으로 생성되거나 지정되어야 합니다.
    /// </remarks>
    Task CreateAsync(T entity);
```

```

/// <summary>
/// 기존 Entity를 비동기적으로 업데이트합니다.
/// </summary>
/// <param name="entity">수정할 Entity 객체</param>
/// <remarks>
/// 주어진 Entity 객체는 반드시 이미 데이터베이스에 존재하는 Entity이어야 하며,
/// 해당 Entity의 식별자는 변경되지 않아야 합니다.
/// 만약 Entity가 존재하지 않으면 예외가 발생할 수 있습니다.
/// </remarks>
Task UpdateAsync(T entity);

/// <summary>
/// Entity를 비동기적으로 삭제합니다.
/// </summary>
/// <param name="entity">삭제할 Entity 객체</param>
/// <remarks>
/// 삭제하려는 Entity는 반드시 데이터베이스에 존재하는 Entity여야 합니다.
/// 만약 해당 Entity가 데이터베이스에 존재하지 않으면 예외가 발생할 수 있습니다.
/// 삭제 후, Entity는 더 이상 데이터베이스에 존재하지 않게 됩니다.
/// </remarks>
Task RemoveAsync(T entity);
}

```

● Repository<T>

/* where T : class, new() -> 제네릭 Type 제약을 정의하는 구문입니다.

이 구문은 제네릭 타입 파라미터 T 에 대해 특정한 제한을 두어, 해당 제네릭 Type 이 특정 조건을 만족하도록 합니다. T -> 제네릭 Type 입니다.

class 제약은 T 가 클래스 타입이어야 합니다.

예를 들어 T 는 int 같은 값 타입이나 struct(구조체)를 사용할 수 없습니다.

대신 T 는 string, MyClass, List<T>와 같은 클래스 타입이어야 합니다.

new() 는 디폴트 생성자가 존재하는 클래스 타입에 대한 제약입니다.

T 는 기본 생성자를 통해 인스턴스를 생성할 수 있는 타입이어야 한다는 조건입니다.

이 조건은 T 가 인수 없이 인스턴스를 생성할 수 있는 생성자를 제공하는

타입이어야 함을 의미합니다. 즉 T 는 매개변수가 없는 생성자가 있어야 합니다. */

```
public class Repository<T> : IRepository<T> where T : class, new()
```

```
{
```

```
    // 데이터베이스 컨텍스트 개체 (DbContext)
```

```
    private readonly DbContext _context;
```

```
    // DbSet은 DbContext로부터 가져오기
```

```
    private DbSet<T> DbSet => _context.Set<T>();
```

```
    /// <summary>
```

```
    /// Repository의 생성자. DbContext를 받아 초기화합니다.
```

```
    /// DbContext 객체는 데이터베이스 연결 및 트랜잭션 관리를 담당합니다.
```

```
    /// </summary>
```

```
    /// <param name="context">DbContext 객체 (데이터베이스 연결 및 트랜잭션 관리)</param>
```

```
    public Repository(DbContext context)
```

```
    {
```

```
        // DbContext 초기화 및 null 체크
```

```
        _context = context ?? throw new ArgumentNullException(nameof(context),  
            "DbContext는 null 일 수 없습니다.});
```

```
    }
```

```

/// <summary>
/// 새로운 Entity를 데이터베이스에 비동기적으로 추가합니다.
/// </summary>
/// <param name="entity">추가할 Entity 객체</param>
public async Task CreateAsync(T? entity)
{
    // Entity가 null일 경우 예외를 발생시킴
    if (entity == null)
        throw new ArgumentNullException(nameof(entity),
            "Entity를 추가하려면 유효한 Entity object가 필요합니다.");

    // 비동기적으로 Entity를 DbSet에 추가
    await DbSet.AddAsync(entity);

    // 데이터베이스에 변경 사항을 저장
    await _context.SaveChangesAsync();
}

/// <summary>
/// 모든 Entity를 비동기적으로 가져옵니다.
/// </summary>
/// <returns>모든 Entity를 담은 컬렉션 (IEnumerable<T>)</returns>
public async Task<IEnumerable<T>> GetAllAsync()
{
    // DbSet에서 모든 Entity를 비동기적으로 가져옴
    return await DbSet.ToListAsync();
}

/// <summary>
/// 특정 ID에 해당하는 Entity를 비동기적으로 가져옵니다.
/// </summary>
/// <param name="id">검색하려는 Entity의 고유 ID</param>
/// <returns>찾은 Entity 또는 null</returns>
public async Task<T?> GetByIdAsync(int id)
{
    // 주어진 ID에 해당하는 Entity를 비동기적으로 찾음
    return await DbSet.FindAsync(id);
}

```

```

/// <summary>
/// 기존 Entity를 데이터베이스에서 비동기적으로 삭제합니다.
/// </summary>
/// <param name="entity">삭제할 Entity 객체</param>
public async Task RemoveAsync(T? entity)
{
    // Entity가 null일 경우 예외를 발생시킴
    if (entity == null)
        throw new ArgumentNullException(nameof(entity),
            "삭제할 Entity는 null 일 수 없습니다.");

    // Entity를 DbSet에서 제거
    DbSet.Remove(entity);

    // 데이터베이스에 변경 사항을 저장
    await _context.SaveChangesAsync();
}

```

```

/// <summary>
/// 기존 Entity를 데이터베이스에서 비동기적으로 업데이트합니다.
/// </summary>
/// <param name="entity">수정할 Entity 객체</param>
public async Task UpdateAsync(T? entity)
{
    // Entity가 null일 경우 예외를 발생시킴
    if (entity == null)
        throw new ArgumentNullException(nameof(entity),
            "Update할 Entity는 null 일 수 없습니다.");

    // Entity를 DbSet에서 업데이트
    DbSet.Update(entity);

    // 데이터베이스에 변경 사항을 저장
    await _context.SaveChangesAsync();
}
}

```

IUnitOfWork & UnitOfWork

● IUnitOfWork 인터페이스 정의

```
/// <summary>
/// 여러 Repository에 대한 트랜잭션 단위 작업을 관리하는 인터페이스
/// </summary>
public interface IUnitOfWork : IDisposable
{
    /// <summary>
    /// Person 엔티티에 대한 Repository
    /// </summary>
    IRepository<Person> PersonRepository { get; }

    /// <summary>
    /// Order 엔티티에 대한 Repository
    /// </summary>
    IRepository<Order> OrderRepository { get; }

    /// <summary>
    /// 모든 변경사항을 하나의 트랜잭션으로 커밋합니다.
    /// </summary>
    Task<int> CommitAsync();

    /// <summary>
    /// 변경사항을 롤백할 필요가 있을 때 사용 (EF에서는 추적된 변경 사항을 취소하는 용도)
    /// </summary>
    void Rollback();
}
```

여기에는 필요한 Entity 를 Repository를 선언하면 됩니다.
(예를 들어 Product, Price, Employee, Customer...)

● UnitOfWork 클래스 구현 예시 (Lazy-Loading 방식 사용, 초기화가 필요한 시점에 로딩)

```
public class UnitOfWork : IUnitOfWork
{
    private readonly DbContext _context;

    // 각 Entity 에 대한 Repository
    public IRepository<Person> PersonRepository { get; }
    public IRepository<Order> OrderRepository { get; }

    public UnitOfWork(DbContext context)
    {
        _context = context ?? throw new ArgumentNullException(nameof(context));
    }

    public IRepository<Person> PersonRepository
        => _personRepository ??= new Repository<Person>(_context);

    public IRepository<Order> OrderRepository
        => _orderRepository ??= new Repository<Order>(_context);

    /// <summary>
    /// 모든 변경사항을 하나의 트랜잭션으로 저장합니다.
    /// </summary>
    public async Task<int> CommitAsync()
    {
        // 트랜잭션을 commit합니다
        return await _context.SaveChangesAsync();
    }

    /// <summary>
    /// 변경사항 롤백 (EF Core에서는 추적된 변경사항을 초기화하는 방식)
    /// </summary>
    public void Rollback()
    {
        // Rollback 처리: Entity 상태 초기화
        foreach (var entry in _context.ChangeTracker.Entries().ToList())
        {
            switch (entry.State)
            {
                {
```

```

        case EntityState.Modified:
            entry.CurrentValues.SetValues(entry.OriginalValues);
            entry.State = EntityState.Unchanged;
            break;
        case EntityState.Added:
            entry.State = EntityState.Detached;
            break;
        case EntityState.Deleted:
            entry.State = EntityState.Unchanged;
            break;
    }
}

// 트랜잭션 롤백 (EF Core 트랜잭션 직접 관리하는 경우)
_context.ChangeTracker.Entries().ToList().ForEach(e => e.Reload());
}

public void Dispose()
{
    // DbContext 리소스 해제
    _context.Dispose();
}
}

```

위 코드는 Lazy 지연 방법 아래 코드는 명시적 초기화 방법

● UnitOfWork 클래스 구현 예시 (즉시 초기화 방식, 생성자에서 모든 Repository 초기화)

```

public class UnitOfWork : IUnitOfWork
{
    private readonly DbContext _context;

    // 각 엔티티에 대한 Repository
    public IRepository<Person> PersonRepository { get; }
    public IRepository<Order> OrderRepository { get; }
    public IRepository<Product> ProductRepository { get; }
    public IRepository<Price> PriceRepository { get; }
    public IRepository<Employee> EmployeeRepository { get; }
    public IRepository<Customer> CustomerRepository { get; }

    public UnitOfWork(DbContext context)
    {
        _context = context;
    }
}

```


// 각 Repository를 초기화

PersonRepository = new Repository<Person>(_context);

OrderRepository = new Repository<Order>(_context);

ProductRepository = new Repository<Product>(_context);

PriceRepository = new Repository<Price>(_context);

EmployeeRepository = new Repository<Employee>(_context);

CustomerRepository = new Repository<Customer>(_context);

}

/// <summary>

/// 트랜잭션 커밋

/// </summary>

public async Task<int> CommitAsync()

{

// 트랜잭션 커밋

return await _context.SaveChangesAsync();

}

/// <summary>

/// 트랜잭션 롤백

/// </summary>

public void Rollback()

{

// EF에서는 변경 사항을 추적하고 롤백하는 기능이 자동으로 제공됩니다.

foreach (var entry in _context.ChangeTracker.Entries())

{

if (entry.State == EntityState.Added)

entry.State = EntityState.Detached;

else if (entry.State == EntityState.Modified)

entry.State = EntityState.Unchanged;

else if (entry.State == EntityState.Deleted)

entry.State = EntityState.Unchanged;

}

}

public void Dispose()

{

// DbContext 리소스 해제

_context?.Dispose();

}

}

● 가독성 비교: Lazy, 생성자 초기화

첫 번째 방식 (Lazy-Loading 방식)

장점:

1. 지연 초기화 (Lazy Initialization):

각 Repository가 실제로 필요한 시점에 생성되므로, 불필요한 개체 생성이 없고 메모리 관리 측면에서 효율적입니다.

예를 들어, PersonRepository나 OrderRepository가 사용되지 않으면 해당 Repository 개체가 생성되지 않기 때문에, 성능상의 이점이 있을 수 있습니다.

2. 클린 코드:

Repository가 실제로 사용될 때만 생성되기 때문에, 이 방식은 초기화 코드가 적고, 초기화 시점이 명확합니다.

단점:

1. 가독성 저하:

Lazy-Loading 방식을 사용하면 코드가 조금 더 복잡해집니다.

??= 연산자와 => 표현식이 추가되어 읽는 사람에게는 다소 혼란스러울 수 있습니다.

Lazy 로딩이 무엇을 의미하는지 잘 모르는 사람에게는 이 코드의 흐름이 직관적이지 않을 수 있습니다.

2. 디버깅:

Repository가 실제로 필요한 시점에 초기화되므로, 디버깅할 때 모든 Repository가 생성된 시점을 추적하기 어렵습니다.

두 번째 방식 (즉시 초기화 방식)

장점:

1. 명확한 초기화:

모든 Repository가 생성자에서 즉시 초기화되므로, 개체의 상태가 명확하고 한 눈에 보기 쉽습니다. 이 방식은 코드의 흐름을 직관적으로 이해하기 쉽습니다.

2. 가독성 우수:

생성자에서 모든 Repository를 초기화하는 방식은 코드가 간단하고 직관적이기 때문에 가독성이 더 높습니다.

Repository를 사용하는 모든 코드에서 각 Repository가 이미 초기화되어 있으므로, 코드를 이해하는 데 어려움이 없습니다.

단점:

1. 메모리 효율성:

생성자에서 모든 Repository를 즉시 초기화하므로, 불필요하게 사용되지 않는 Repository 개체들이 메모리에 상주할 수 있습니다.

예를 들어, ProductRepository가 전혀 사용되지 않더라도, 생성자에서 초기화되므로 메모리 낭비가 있을 수 있습니다.

2. 확장성 제한:

새로운 Repository를 추가할 때마다 생성자에 수정이 필요하고, 초기화하는 코드가 길어지므로 코드의 확장성 측면에서 조금 더 관리가 어려워질 수 있습니다.

결론:

****두 번째 방식 (즉시 초기화 방식)**이 가독성 측면에서 더 우수하다고 볼 수 있습니다.**

명확한 코드 흐름과 직관적인 초기화 덕분에 코드가 간단하고 읽기 쉬우며, 초보 개발자나 팀원들이 쉽게 이해할 수 있습니다.

비록 모든 Repository를 한 번에 초기화하기 때문에 메모리 사용 측면에서 비효율적일 수 있지만, 실제 프로젝트에서 성능이 큰 문제가 되지 않으면 가독성의 장점이 더 우선시 될 수 있습니다.

따라서 가독성에 최우선 순위를 둔다면 두 번째 방식이 더 나은 선택입니다

● IUserService 클래스

```
public interface IUserService
{
    Task<IEnumerable<Person>> GetAllUsersAsync();
    Task<Person?> GetUserByIdAsync(int id);
    Task AddUserAsync(Person person);
    Task UpdateUserAsync(Person person);
    Task DeleteUserAsync(Person person);
}
```

● UserService 클래스 (CRUD 포함)

UserService는 IUnitOfWork를 사용하여 CRUD 작업을 처리하는 서비스 레이어입니다. 여기서 Person에 대한 CRUD 기능을 포함하고 있습니다

```
public class UserService : IUserService
{

```

```

private readonly IUnitOfWork _unitOfWork;

public UserService(IUnitOfWork unitOfWork)
{
    _unitOfWork = unitOfWork;
}

public async Task<IEnumerable<Person>> GetAllUsersAsync()
    => await _unitOfWork.PersonRepository.GetAllAsync();

public async Task<Person?> GetUserByIdAsync(int id)
    => await _unitOfWork.PersonRepository.GetByIdAsync(id);

public async Task AddUserAsync(Person person)
{
    try
    {
        if (person == null) throw new ArgumentNullException(nameof(person),
            "Person은 null일 수 없습니다.");

        await _unitOfWork.PersonRepository.CreateAsync(person);
        await _unitOfWork.CommitAsync();
    }
    catch (Exception ex)
    {
        // 로깅 예시: logError(ex); // 로깅 기능을 추가
        throw new ApplicationException("User를 삭제하는 동안 오류가 발생했습니다", ex);
    }
}

public async Task UpdateUserAsync(Person person)
{
    await _unitOfWork.PersonRepository.UpdateAsync(person);
    await _unitOfWork.CommitAsync();
}

public async Task DeleteUserAsync(Person person)
{
    try
    {
        if (person == null) throw new ArgumentNullException(nameof(person),

```

```

        "Person은 null 일수 없습니다..");

        await _unitOfWork.PersonRepository.RemoveAsync(person);
        await _unitOfWork.CommitAsync();
    }
    catch (Exception ex)
    {
        // logError(ex); // 로깅 추가
        throw new ApplicationException
            ("User를 삭제하는 동안 오류가 발생했습니다.", ex);
    }
}
}

```

UserService 클래스는 CRUD 작업을 처리하는 서비스 레이어로서, 비즈니스 로직을 담당합니다.

서비스 레이어는 **UnitOfWork**를 통해 **PersonRepository**에 접근하여 데이터베이스의 데이터를 조작하고, **CommitAsync()**를 호출하여 변경사항을 저장합니다. 이 구조는 **SRP(단일 책임 원칙)**과 **DI(의존성 주입)** 원칙을 따르며, 계층형 아키텍처에서 흔히 사용되는 패턴입니다

서비스 레이어에서 처리하는 CRUD

1. GetAllUsersAsync:

데이터베이스에서 모든 Person 객체를 가져옵니다.

UnitOfWork의 PersonRepository를 사용하여 읽기 작업을 수행합니다.

2. GetUserByIdAsync:

특정 ID에 해당하는 Person을 데이터베이스에서 비동기적으로 가져옵니다.

역시 PersonRepository.GetByIdAsync()를 호출하여 읽기 작업을 처리합니다.

3. AddUserAsync:

새 Person 객체를 추가합니다.

먼저 ArgumentNullException을 던져 person이 null이 아닌지 체크하고, CreateAsync()를 호출하여 생성 작업을 처리합니다.

이후 CommitAsync()로 저장합니다.

4. UpdateUserAsync:

기존의 Person 객체를 수정합니다.

UpdateAsync()로 수정하고, CommitAsync()로 DB에 반영합니다.

5. DeleteUserAsync:

Person 객체를 삭제합니다.

마찬가지로 ArgumentNullException으로 null 체크 후,
RemoveAsync()를 호출하여 삭제 작업을 처리합니다.

이후 CommitAsync()로 삭제 반영을 합니다.

● ViewModel 사용 예시

UserService 에서 CRUD를 사용하는 예시입니다

```
public class MainViewModel : ViewModelBase
{
    private readonly IUserService _userService;
    public ObservableCollection<Person> People { get; } = new();
    public DelegateCommand _loadUsersCommand;
    public DelegateCommand LoadUsersCommand => _loadUsersCommand
        ??= new DelegateCommand(async () => await LoadUsersAsync());
    public DelegateCommand _addUserCommand;
    public DelegateCommand AddUserCommand => _addUserCommand
        ??= new DelegateCommand(async () => await AddUserAsync());
    public DelegateCommand<Person> _editUserCommand;
    public DelegateCommand<Person> EditUserCommand => _editUserCommand
        ??= new DelegateCommand(async (person) => await EditUserAsync(person));
    public DelegateCommand<Person> _deleteUserCommand;
    public DelegateCommand<Person> DeleteUserCommand => _deleteUserCommand
        ??= new DelegateCommand(async (person) => await DeleteUserAsync(person));
    public UserListViewModel(IUserService userService)
    {
        _userService = userService;
    }
    // 읽기
    private async Task LoadUsersAsync()
```

```

{
    var users = await _userService.GetAllUsersAsync();
    People.Clear();
    foreach (var user in users)
        People.Add(user);
}

```

// 추가하기

```

private async Task AddUserAsync()
{
    var newPerson = new Person
    {
        Name = "새 사용자",
        Age = 0,
        BirthDay = DateTime.Now,
        Address = "입력 필요"
    };

    await _userService.AddUserAsync(newPerson);
    await LoadUsersAsync();
}

```

// 편집하기

```

private async Task EditUserAsync(Person person)
{
    if (person == null) return;

    person.Name = "수정된 이름";
    await _userService.UpdateUserAsync(person);
    await LoadUsersAsync();
}

```

// 삭제하기

```

private async Task DeleteUserAsync(Person person)
{
    if (person == null) return;

    await _userService.DeleteUserAsync(person);
    People.Remove(person);
}
}

```

위 DelegateCommand 는 Lazy 방식으로 1~2개, 간단한 구성일 경우 추천 Command 가 많고 복잡한 ViewModel 은 생성자 명시 초기화 방식 권장

● Lazy 방식 커맨드란?

- 보통 커맨드 필드를 즉시 생성하지 않고, 필요할 때 생성하는 방식입니다.
- 이 방식으로 불필요한 메모리 할당을 줄이고, 초기화 비용을 아낄 수 있습니다.
- 일반적으로 커맨드 프로퍼티에 private DelegateCommand? _myCommand; 필드를 두고, 프로퍼티 내부에서 ?? = 연산자로 최초 접근 시에만 생성하는 형태입니다.

```
<Button Content="불러오기" Command="{Binding LoadUsersCommand}" />
```

```
<Button Content="추가" Command="{Binding AddUserCommand}" />
```

```
<ListBox ItemsSource="{Binding People}" SelectedItem="{Binding SelectedPerson}">
```

```
<ListBox.ItemTemplate>
```

```
<DataTemplate>
```

```
<StackPanel Orientation="Horizontal">
```

```
<TextBlock Text="{Binding Name}" />
```

```
<Button Content="수정"
```

```
Command="{Binding DataContext.EditUserCommand,
```

```
RelativeSource={RelativeSource AncestorType=UserControl}}"
```

```
CommandParameter="{Binding}" />
```

```
<Button Content="삭제"
```

```
Command="{Binding DataContext.DeleteUserCommand,
```

```
RelativeSource={RelativeSource AncestorType=UserControl}}"
```

```
CommandParameter="{Binding}" />
```

```
</StackPanel>
```

```
</DataTemplate>
```

```
</ListBox.ItemTemplate>
```

```
</ListBox>
```

● Prism DI 등록

```
protected override void RegisterTypes(IContainerRegistry containerRegistry)
{
```

```
    // IUserService 및 IUnitOfWork를 각각 DI 컨테이너에 등록합니다.
```

```
    containerRegistry.Register<IUserService, UserService>();
```

```
    containerRegistry.Register<IUnitOfWork, UnitOfWork>();
```



```
// Repository를 제네릭으로 등록하여 다양한 엔티티에 대해 사용할 수 있게 합니다.
containerRegistry.Register(typeof(IRepository<>), typeof(Repository<>));

// ApplicationDbContext를 등록합니다. DB 연결 및 트랜잭션을 처리하는 중요한 부분입니다.
containerRegistry.Register<AppDbContext>();

}
```

● 이 구조의 장점

장점	설명
관심사 분리	ViewModel, Service, Repository 분리
테스트 용이	각 계층을 Mock으로 대체 가능
재사용성 증가	서비스 계층은 콘솔, 웹 등 다른 환경에서도 사용 가능
유지보수 용이	변경이 필요한 로직만 수정 가능
MVVM 패턴에 적합	ViewModel은 UI만 책임지고 로직은 Service에 위임

● 기타

if 문 사용 권장, 예측할 수 없는 예외나 외부 시스템과의 오류는 try-catch 문 사용

try-catch 문 사용 권장

1. 외부 API 호출 시 발생할 수 있는 예외

외부 시스템이나 API와의 연결은 항상 성공하지 않을 수 있습니다.
네트워크 문제나 API 서버의 다운 등 예기치 못한 오류가 발생할 수 있기 때문에
try-catch를 사용하여 안전하게 예외를 처리해야 합니다.

```
public async Task<string> GetDataFromApiAsync(string endpoint)
{
    try
    {
        // 외부 API 호출
        var response = await _httpClient.GetStringAsync(endpoint);
        return response;
    }
    catch (HttpRequestException ex)
    {
        // 네트워크 문제나 잘못된 URL 등 예외 처리
        throw new ApplicationException("API 호출 중 문제가 발생했습니다.", ex);
    }
}
```

```

catch (TimeoutException ex)
{
    // 네트워크 타임아웃 예외 처리
    throw new ApplicationException("API 요청 시간이 초과되었습니다.", ex);
}
catch (Exception ex)
{
    // 예상치 못한 예외를 처리
    throw new ApplicationException("알 수 없는 오류가 발생했습니다.", ex);
}
}

```

HttpRequestException : 네트워크 요청에 문제가 있을 경우 발생
(예: 잘못된 URL, 네트워크 연결 실패 등).

TimeoutException : 요청이 일정 시간 동안 응답이 없을 경우 발생.

이와 같은 예외는 사전에 예상할 수 없거나 외부 환경에 의존적이기 때문에 try-catch 를 사용해 안전하게 처리합니다.

2. 파일 읽기/쓰기 작업

파일 시스템에 접근할 때도 예기치 못한 오류가 발생할 수 있습니다.

예를 들어, 파일이 존재하지 않거나, 권한이 없거나,
디스크에 문제가 있을 수 있습니다.

```

public async Task<string> ReadFileAsync(string filePath)
{
    try
    {
        // 파일 읽기
        var fileContent = await File.ReadAllTextAsync(filePath);
        return fileContent;
    }
    catch (FileNotFoundException ex)
    {
        // 파일이 존재하지 않을 경우 예외 처리
        throw new ApplicationException("지정된 파일을 찾을 수 없습니다.", ex);
    }
    catch (UnauthorizedAccessException ex)

```

```

{
    // 파일에 접근할 권한이 없을 경우 예외 처리
    throw new ApplicationException("파일에 접근할 수 없습니다. 권한을 확인하세요.", ex);
}
catch (IOException ex)
{
    // I/O 오류 처리
    throw new ApplicationException("파일 읽기 중 오류가 발생했습니다.", ex);
}
catch (Exception ex)
{
    // 예기치 못한 다른 예외 처리
    throw new ApplicationException("알 수 없는 오류가 발생했습니다.", ex);
}
}

```

FileNotFoundException : 파일이 존재하지 않으면 발생.

UnauthorizedAccessException : 파일에 대한 접근 권한이 없으면 발생.

IOException : 파일을 읽는 도중에 I/O 오류가 발생하면 발생.

이런 상황도 예기치 못한 오류가 발생할 수 있기 때문에,
try-catch를 사용하여 처리하는 것이 적절합니다.

3. 데이터베이스 연결 오류

데이터베이스와의 연결도 예기치 못한 오류가 발생할 수 있는 경우입니다.

예를 들어, 네트워크 연결 문제나 데이터베이스 서버가 응답하지 않는
상황에서 발생할 수 있습니다.

```

public async Task<IEnumerable<Customer>> GetCustomersAsync()
{
    try
    {
        // 데이터베이스 쿼리 실행
        return await _dbContext.Customers.ToListAsync();
    }
    catch (SqlException ex)
    {
        // SQL 오류 (예: 연결 실패, 쿼리 오류 등)
    }
}

```

```

        throw new ApplicationException("DB 연결 또는 쿼리 실행 중 오류가 발생했습니다.", ex);
    }
    catch (TimeoutException ex)
    {
        // 데이터베이스 연결 시간 초과 오류
        throw new ApplicationException("DB 연결 시간이 초과되었습니다.", ex);
    }
    catch (Exception ex)
    {
        // 예기치 못한 다른 오류
        throw new ApplicationException("알 수 없는 오류가 발생했습니다.", ex);
    }
}

```

SQLException : DB 연결 오류, 쿼리 오류 등 SQL 관련 오류가 발생하면 발생.

TimeoutException : DB 연결이 시간 초과로 실패한 경우 발생.

데이터베이스와의 연결은 외부 시스템에 의존하기 때문에 예기치 못한 오류가 발생할 수 있으므로 try-catch로 처리하는 것이 중요합니다.

4. 외부 서비스와의 파일 업로드/다운로드

파일 업로드나 다운로드 시에도 여러 가지 예기치 못한 오류가 발생할 수 있습니다. 예를 들어, 파일 크기 제한 초과, 네트워크 오류 등이 있을 수 있습니다.

```

public async Task UploadFileAsync(IFormFile file)
{
    try
    {
        if (file.Length > 10 * 1024 * 1024) // 10MB 이상은 제한
        {
            throw new InvalidOperationException("파일 크기가 너무 큼니다.");
        }

        using (var stream = new FileStream("path_to_upload_directory", FileMode.Create))
        {
            await file.CopyToAsync(stream);
        }
    }
}

```

```

catch (InvalidOperationException ex)
{
    // 비즈니스 로직에서 정의된 오류
    throw new ApplicationException("파일 크기 제한을 초과했습니다.", ex);
}
catch (IOException ex)
{
    // I/O 오류 처리 (디스크 공간 부족 등)
    throw new ApplicationException("파일을 업로드하는 동안 오류가 발생했습니다.", ex);
}
catch (Exception ex)
{
    // 그 외 모든 예외
    throw new ApplicationException("알 수 없는 오류가 발생했습니다.", ex);
}
}

```

InvalidOperationException : 비즈니스 로직에 의한 예외 (파일 크기 제한 등).

IOException : 디스크 공간 부족, 권한 문제 등 파일 시스템 관련 오류.

Exception : 기타 예상치 못한 오류.

외부 파일 시스템에 접근할 때에도 네트워크 문제, 디스크 공간 부족, 파일 시스템 권한 등의 이유로 예기치 않은 오류가 발생할 수 있기 때문에 try-catch로 처리하는 것이 필요합니다.

결론:

try-catch는 외부 시스템과의 상호작용이나 예측할 수 없는 오류가 발생할 가능성이 있는 상황에서 안전하게 예외를 처리하고 복구할 수 있는 방법을 제공합니다

예를 들어 외부 API 호출, 데이터베이스 연결, 파일 읽기/쓰기, 외부 서비스와의 파일 업로드/다운로드 등에서는 예기치 못한 오류나 외부 시스템의 문제로 인해 예외가 발생할 수 있으므로 try-catch로 감싸서 적절히 예외를 처리하는 것이 중요합니다

개선사항

1. 로깅 (Logging)

예외 처리가 잘 되어 있으나, 실제 운영 환경에서는 로깅이 중요합니다.

`logError()`를 통해 예외를 기록하는 방법을 사용하셨다고 언급하셨는데, 이를 실제 로깅 시스템과 연동하는 방식도 고려할 수 있습니다.

예를 들어, **NLog**, **Serilog**, **log4net**과 같은 라이브러리로 로깅을 설정하고, 에러 메시지와 예외 스택 추적 등을 기록하는 것이 중요합니다.

```
try
{
    // 비즈니스 로직
}
catch (Exception ex)
{
    _logger.LogError(ex, "User를 추가하는 동안 오류가 발생했습니다.");
    throw new ApplicationException("User를 추가하는 동안 오류가 발생했습니다.", ex);
}
```

이와 같은 방식으로 로그를 남기면 문제 발생 시 디버깅이나 모니터링에 큰 도움이 됩니다.

2. 유효성 검사 (Validation)

유효성 검사는 서비스 계층에서 필수적인 부분입니다.

예를 들어 **AddUserAsync**와 **UpdateUserAsync** 메서드에서 유효성 검사를 강화할 수 있습니다.

Person 개체의 속성들을 검증하여 잘못된 데이터가 들어오는 것을 방지할 수 있습니다.

```
public async Task AddUserAsync(Person person)
{
    if (person == null) throw new ArgumentNullException(nameof(person), "Person null일 수 없습니다.");
    if (string.IsNullOrEmpty(person.Name)) throw new ArgumentException("Name은 필수입니다.");
    if (person.Age < 18) throw new ArgumentException("User는 18세 이상이어야 합니다.");

    await _unitOfWork.PersonRepository.CreateAsync(person);
    await _unitOfWork.CommitAsync();
}
```

FluentValidation 과 같은 라이브러리를 사용하여 복잡한 유효성 검사를 분리하는 것도 좋은 방법입니다.

3. 트랜잭션 관리 (Transaction Management)

UnitOfWork를 사용하여 트랜잭션을 관리하고 있습니다.

이는 여러 저장소에 대한 작업을 원자적으로 처리하는 좋은 방식입니다.

그러나 추가적으로 고려할 수 있는 사항은 트랜잭션의 세분화와 롤백 정책입니다.

예를 들어, 중첩된 트랜잭션이 필요한 경우, 별도로 트랜잭션을 관리하거나, 특정 작업에 대해서만 롤백을 적용할 수 있는 로직을 추가할 수 있습니다.

```
public async Task<int> CommitAsync()
{
    using var transaction = await _context.Database.BeginTransactionAsync();
    try
    {
        var result = await _context.SaveChangesAsync();
        await transaction.CommitAsync();
        return result;
    }
    catch (Exception ex)
    {
        await transaction.RollbackAsync();
        _logger.LogError(ex, "Commit 실패");
        throw;
    }
}
```

위와 같은 방식으로 트랜잭션을 명시적으로 관리할 수 있으며, 트랜잭션 실패 시 롤백이 발생합니다.

4. 에러 메시지와 예외 계층화 (Error Handling Hierarchy)

ApplicationException 외에도 커스텀 예외를 정의하여, 비즈니스 로직에서 발생할 수 있는 예외를 명확하게 구분하는 것이 좋습니다.

예를 들어, UserNotFoundException, InvalidUserException과 같은 사용자 정의 예외를 만들어서 예외 계층을 구성할 수 있습니다.

```

public class UserNotFoundException : ApplicationException
{
    public UserNotFoundException(string message) : base(message) { }
}

public async Task<Person?> GetUserByIdAsync(int id)
{
    var person = await _unitOfWork.PersonRepository.GetByIdAsync(id);
    if (person == null) throw new UserNotFoundException($"ID가 {id}인 사용자를 찾을 수 없습니다.");
    return person;
}

```

이런 방식으로 예외를 계층화하면, 예외를 보다 구체적으로 처리할 수 있습니다.

5. 비즈니스 규칙 적용 (Business Rule Enforcement)

CRUD 작업에서 비즈니스 규칙을 추가하여,
데이터 변경에 대해 조건을 명확히 정의할 수 있습니다.

예를 들어, Person 엔티티에서 사용자가 삭제되기 전에 관련된
다른 엔티티(예: Order, Invoice 등)가 없는지 검증하는 로직을 추가할 수 있습니다.

```

public async Task DeleteUserAsync(Person person)
{
    if (person == null) throw new ArgumentNullException(
        nameof(person), "Person null일 수 없습니다.");

    // 비즈니스 규칙: 사용자가 관련된 주문이 있으면 삭제할 수 없음
    var orders = await _unitOfWork.OrderRepository
        .GetByConditionAsync(o => o.PersonId == person.Id);

    if (orders.Any()) throw new InvalidOperationException(
        "관련 주문이 있으므로 사용자를 삭제할 수 없습니다.");

    await _unitOfWork.PersonRepository.RemoveAsync(person);
    await _unitOfWork.CommitAsync();
}

```

이렇게 비즈니스 규칙을 서비스 계층에서 구현하면,
애플리케이션의 무결성을 유지할 수 있습니다.

6. 상태 관리 (State Management)

Person 객체나 다른 엔티티에 대해 상태 변경 추적을 강화할 수 있습니다.

예를 들어, 객체의 상태가 Added, Modified, Deleted 중 무엇인지 명확히 추적하고, 각 상태에 맞는 추가적인 비즈니스 로직을 실행할 수 있습니다.

```
public async Task UpdateUserAsync(Person person)
{
    if (person == null) throw new ArgumentNullException(nameof(person));

    var existingUser = await _unitOfWork.PersonRepository.GetByldAsync(person.Id);
    if (existingUser == null) throw new UserNotFoundException("User를 찾을 수 없습니다.");

    // 상태 추적
    _context.Entry(person).State = EntityState.Modified;
    await _unitOfWork.CommitAsync();
}
```

상태를 명시적으로 처리하면, 잘못된 상태의 객체가 데이터베이스에 반영되는 일을 방지할 수 있습니다.

결론

추가적인 개선 포인트는 다음과 같습니다:

- 1. 로깅** : 예외가 발생하면 로깅을 통해 에러를 추적하고, 이후의 디버깅이나 문제 해결에 도움이 되도록 합니다.
- 2. 유효성 검사** : 비즈니스 규칙에 맞는 유효성 검사를 추가하여 잘못된 데이터가 입력되는 것을 방지합니다.
- 3. 트랜잭션 관리** : 명시적으로 트랜잭션을 관리하여 여러 레포지토리 작업을 하나의 트랜잭션으로 묶을 수 있습니다.
- 4. 커스텀 예외** : 보다 구체적인 예외 클래스를 정의하여, 예외를 세분화하고 처리할 수 있습니다.
- 5. 비즈니스 규칙** : 데이터 변경 시 비즈니스 규칙을 적용하여, 상태를 검증하거나 유효성 검사를 강화합니다.
- 6. 상태 관리** : 객체의 상태를 명확히 추적하고 처리하여, 불필요한 오류를 방지할 수 있습니다.

이런 방식으로 개선하면, 코드가 더 유연하고 확장 가능하며, 가독성도 높아집니다.